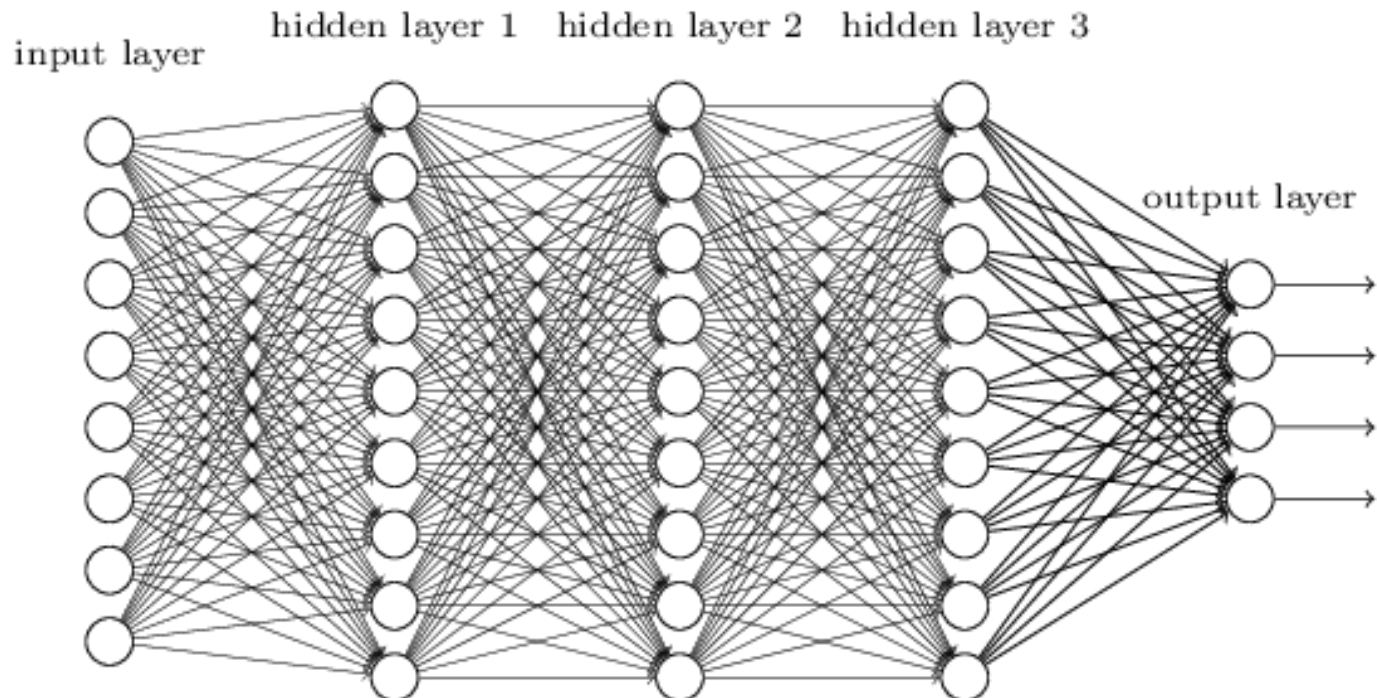


Convolutional Neural Networks

Convolutional Neural Network: CNN

- We know it is good to learn a small model.
- From this fully connected model, do we really need all the edges?
- Can some of these be shared?



Consider learning an image:

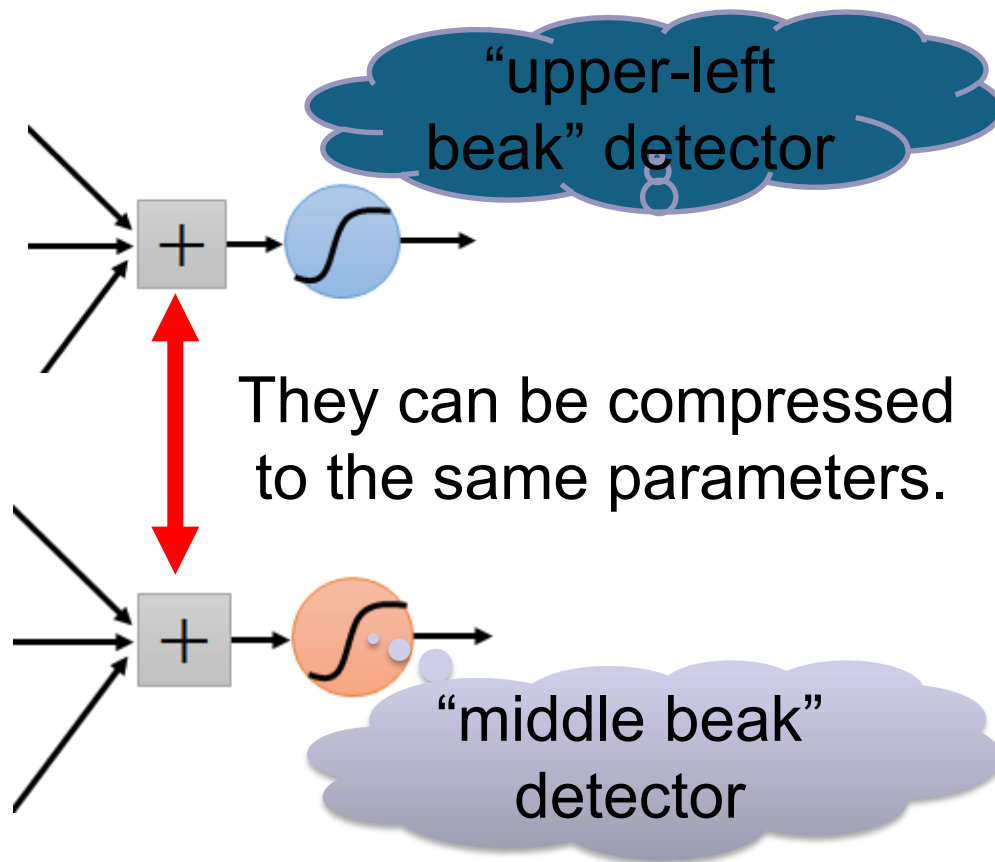
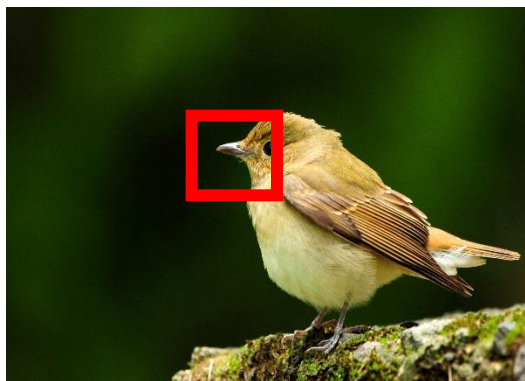
- Some patterns are much smaller than the whole image

Can represent a small region with fewer parameters



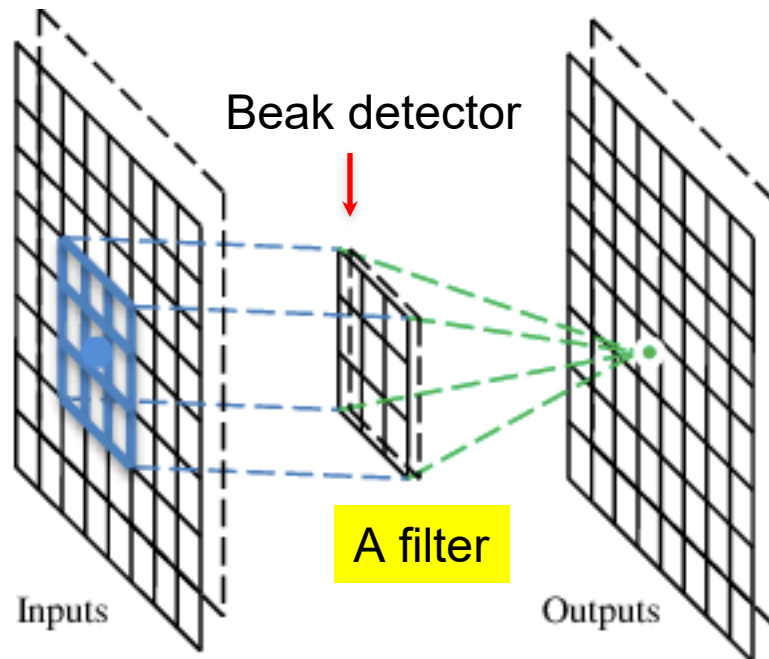
Same pattern appears in different places:
They can be compressed!

What about training a lot of such “small” detectors
and each detector must “move around”.



A convolutional layer

A CNN is a neural network with some convolutional layers (and some other layers). A convolutional layer has a number of filters that does convolutional operation.



Convolution

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

Convolution

stride=

1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Dot
product



3

-1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Convolution

If

stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

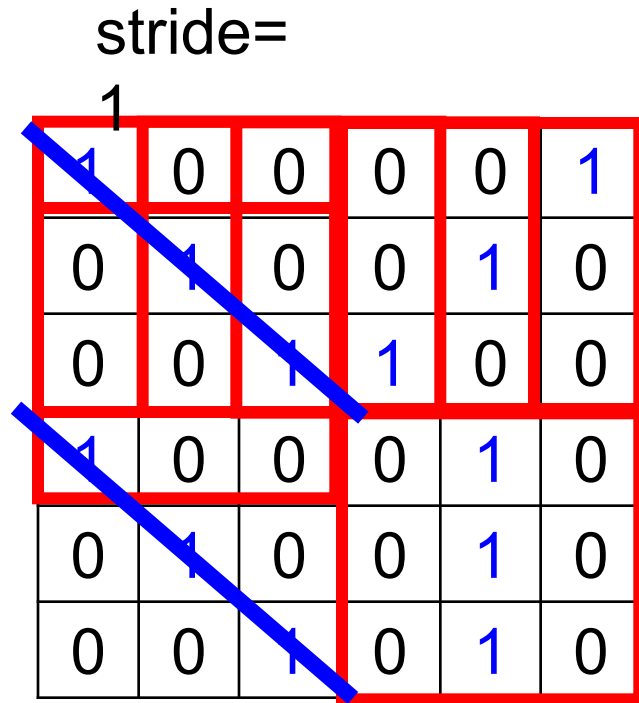
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

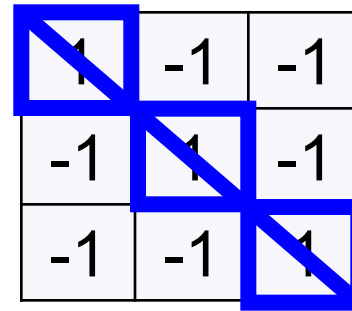
3

-3

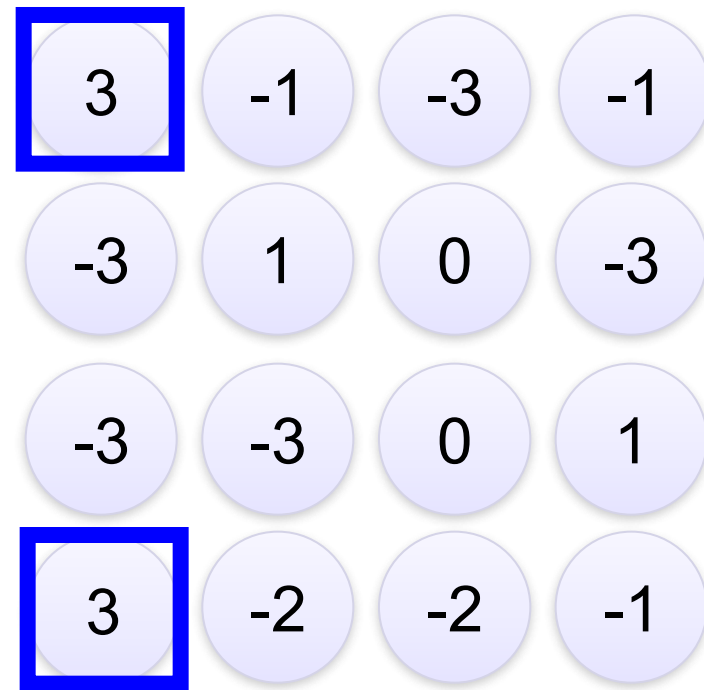
Convolution



6 x 6 image



Filter 1



Convolution

stride=

1

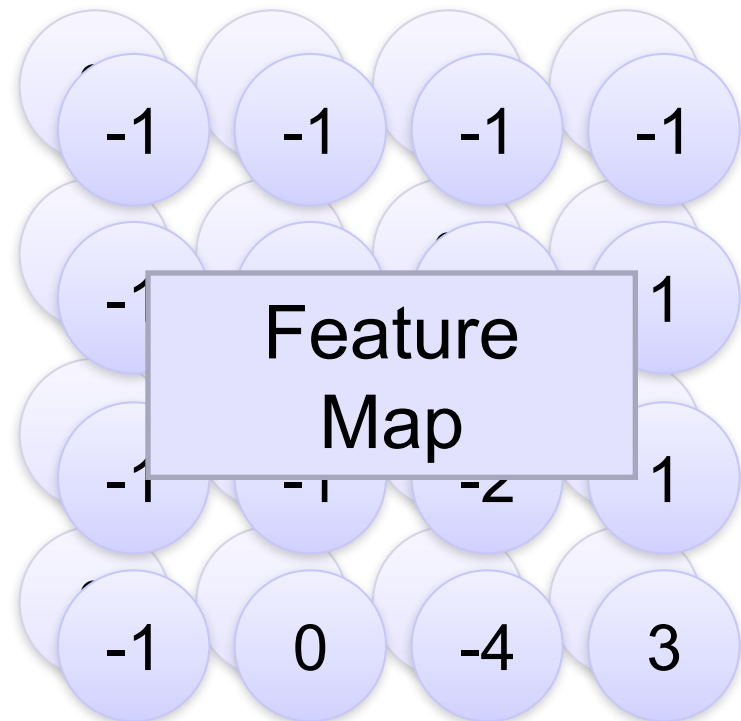
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

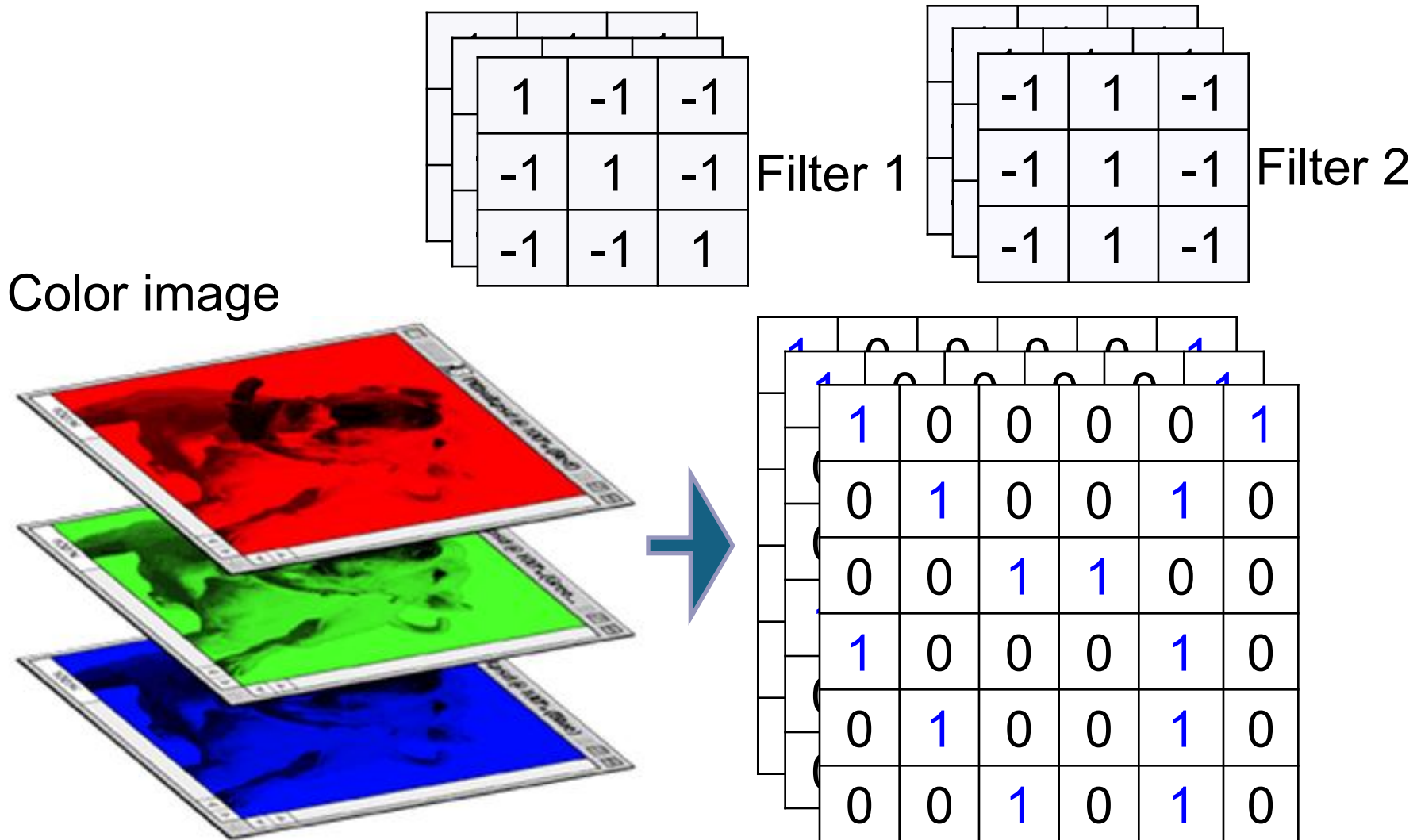
Filter 2

Repeat this for each filter



Two 4 x 4 images
Forming 2 x 4 x 4 matrix

Color image: RGB 3 channels



Convolution v.s. Fully Connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

image

1	-1	-1
-1	1	-1
-1	-1	1

-1	1	-1
-1	1	-1
-1	1	-1

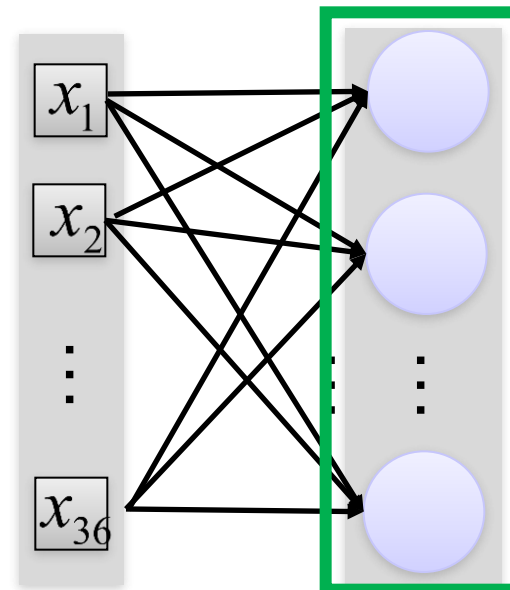


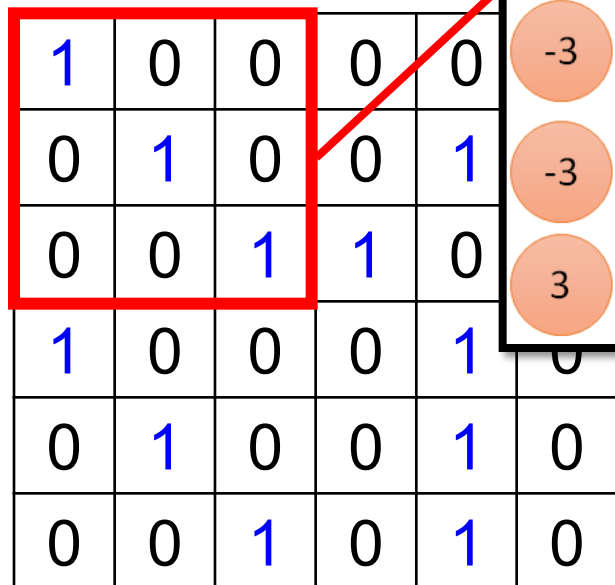
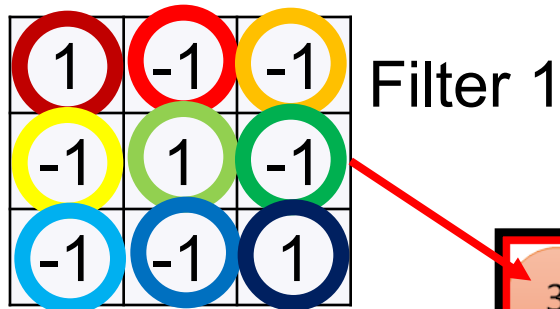
convolution

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Fully-
connected

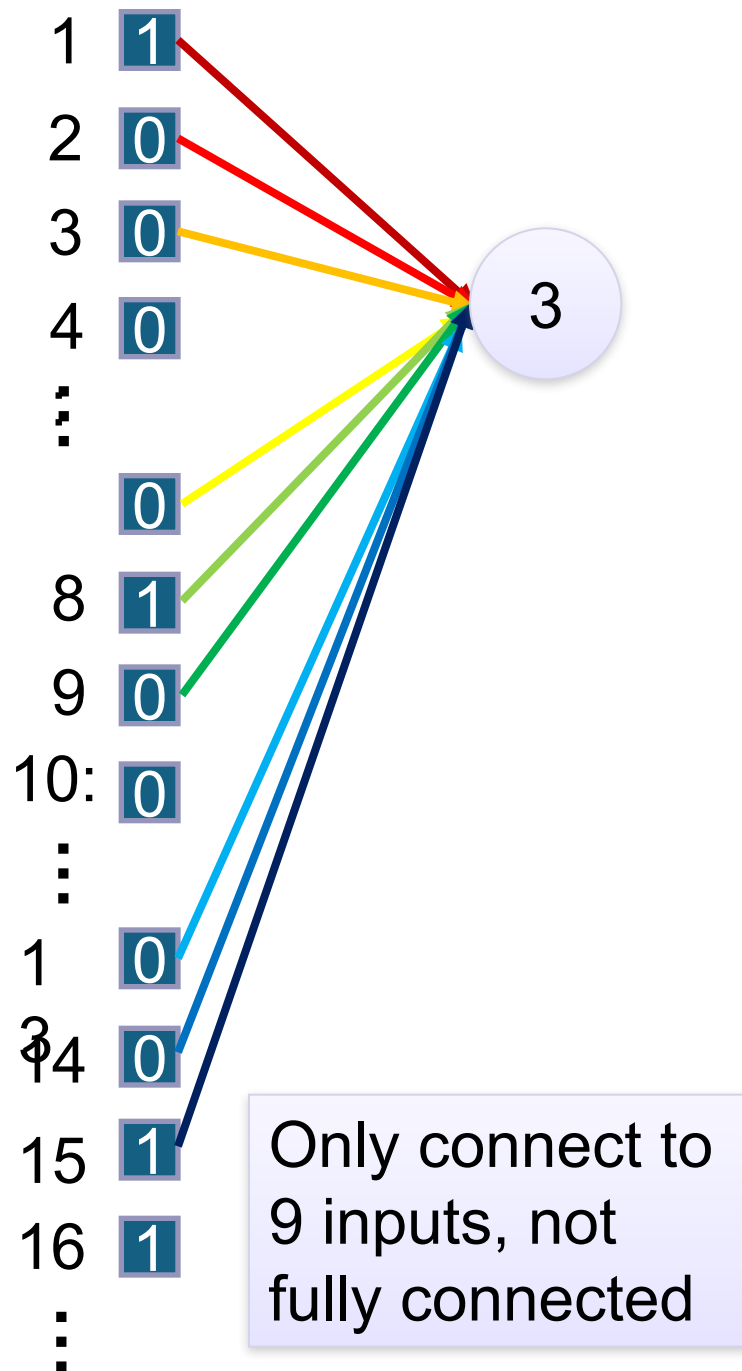
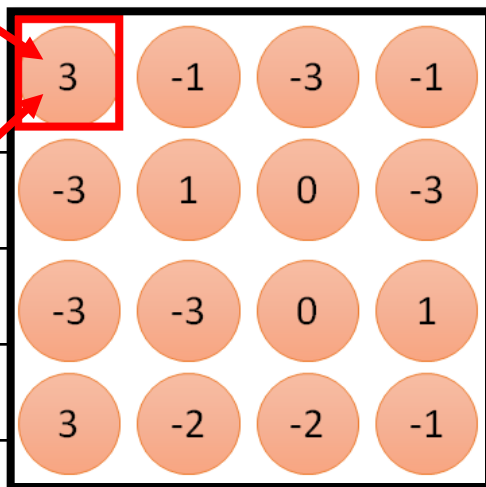
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

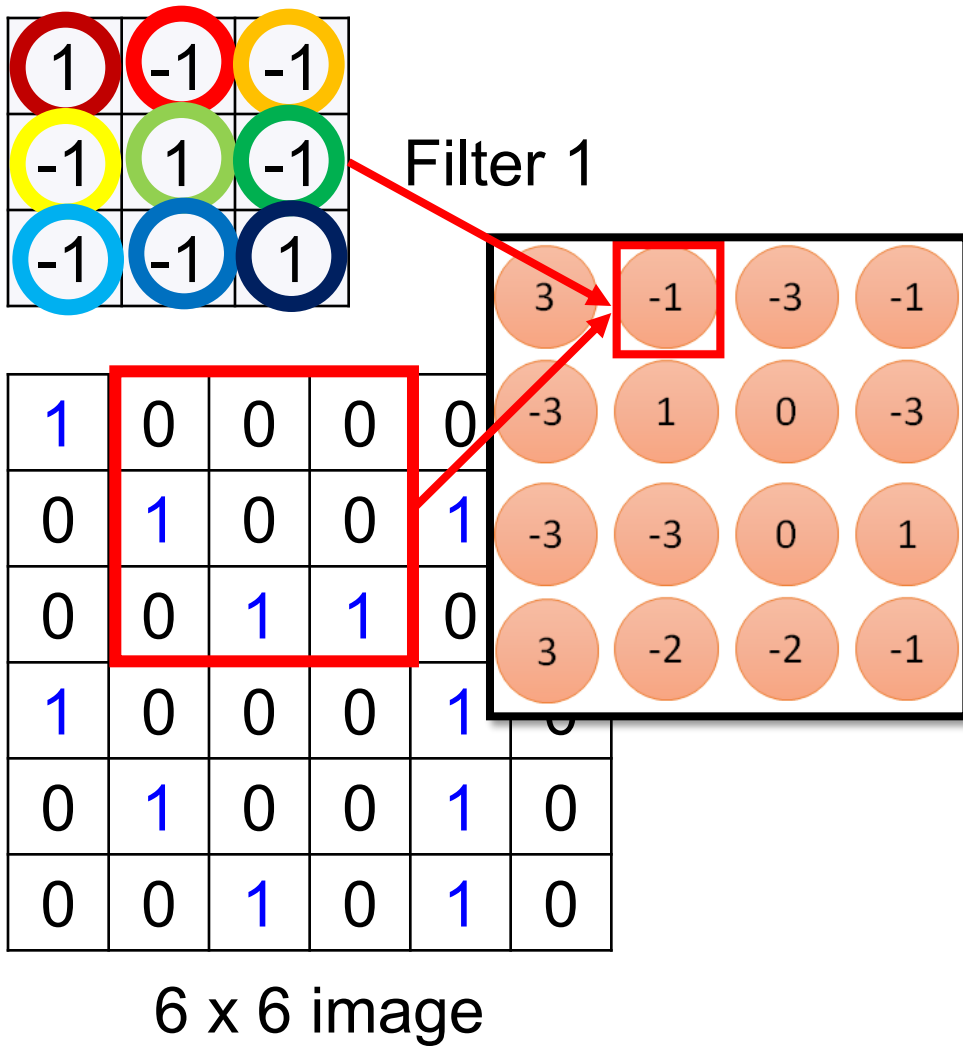




6 x 6 image

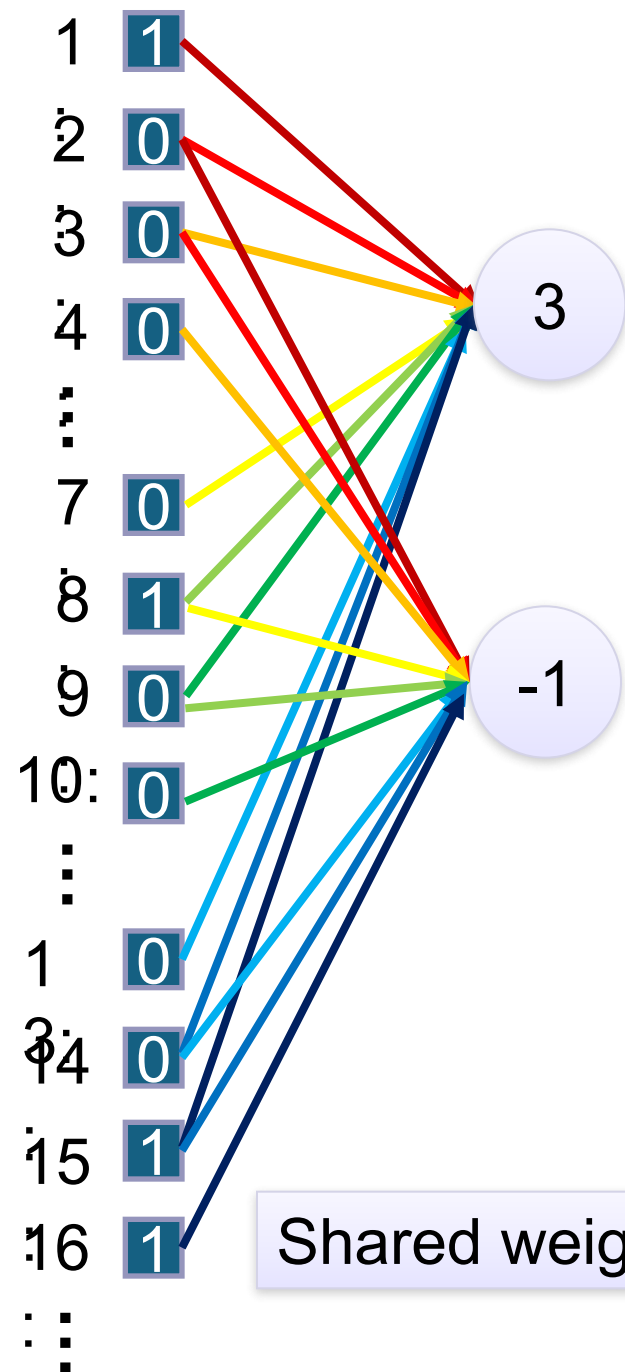
fewer parameters!





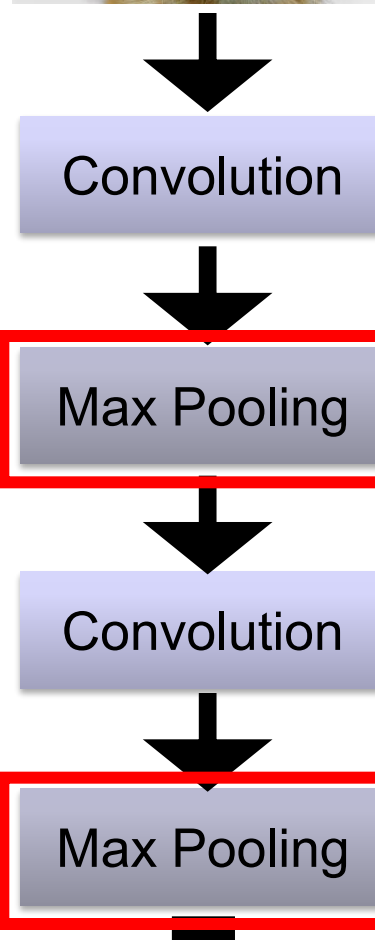
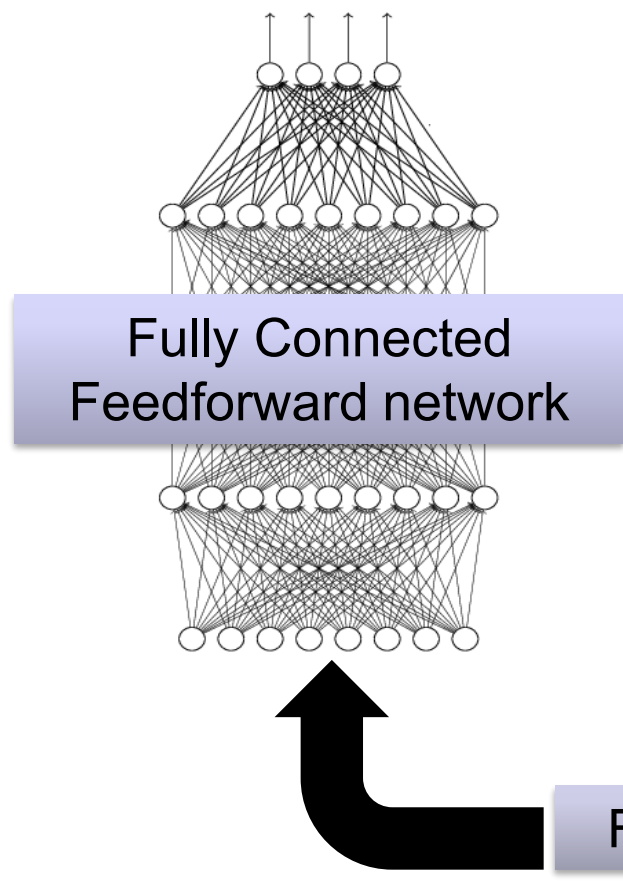
Fewer parameters

Even fewer parameters



The whole CNN

cat dog



Can
repeat
many
times

Flattened

Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Why Pooling

- Subsampling pixels will not change the object

bird



Subsampling

bird



We can subsample the pixels to make image
smaller → fewer parameters to characterize the image

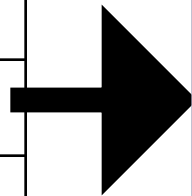
A CNN compresses a fully connected network in two ways:

- Reducing number of connections
- Shared weights on the edges
- Max pooling further reduces the complexity

Max Pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

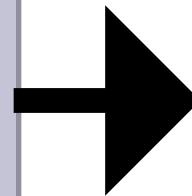
6 x 6 image



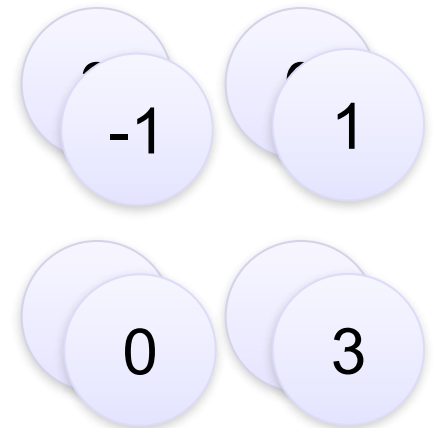
Conv



Max
Pooling



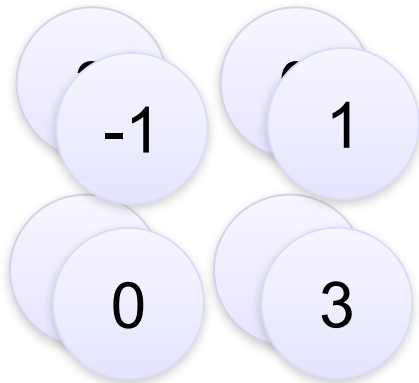
New image
but smaller



2 x 2 image

Each filter
is a channel

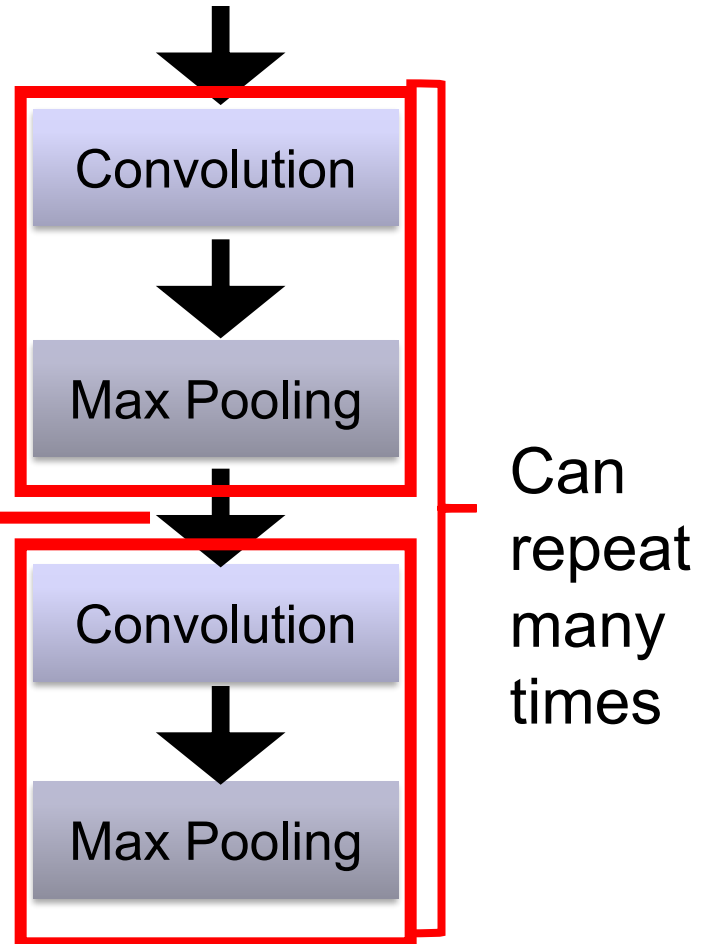
The whole CNN



A new image

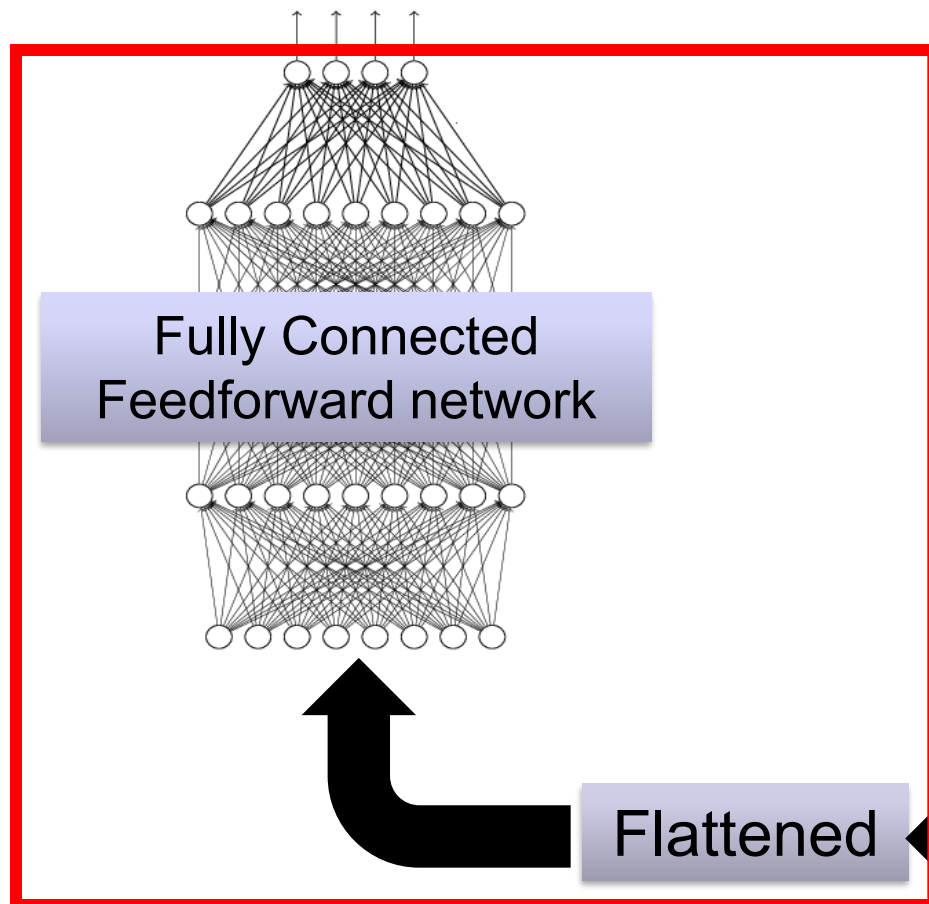
Smaller than the original image

The number of channels is the number of filters



The whole CNN

cat dog



Convolution

Max Pooling

A new image

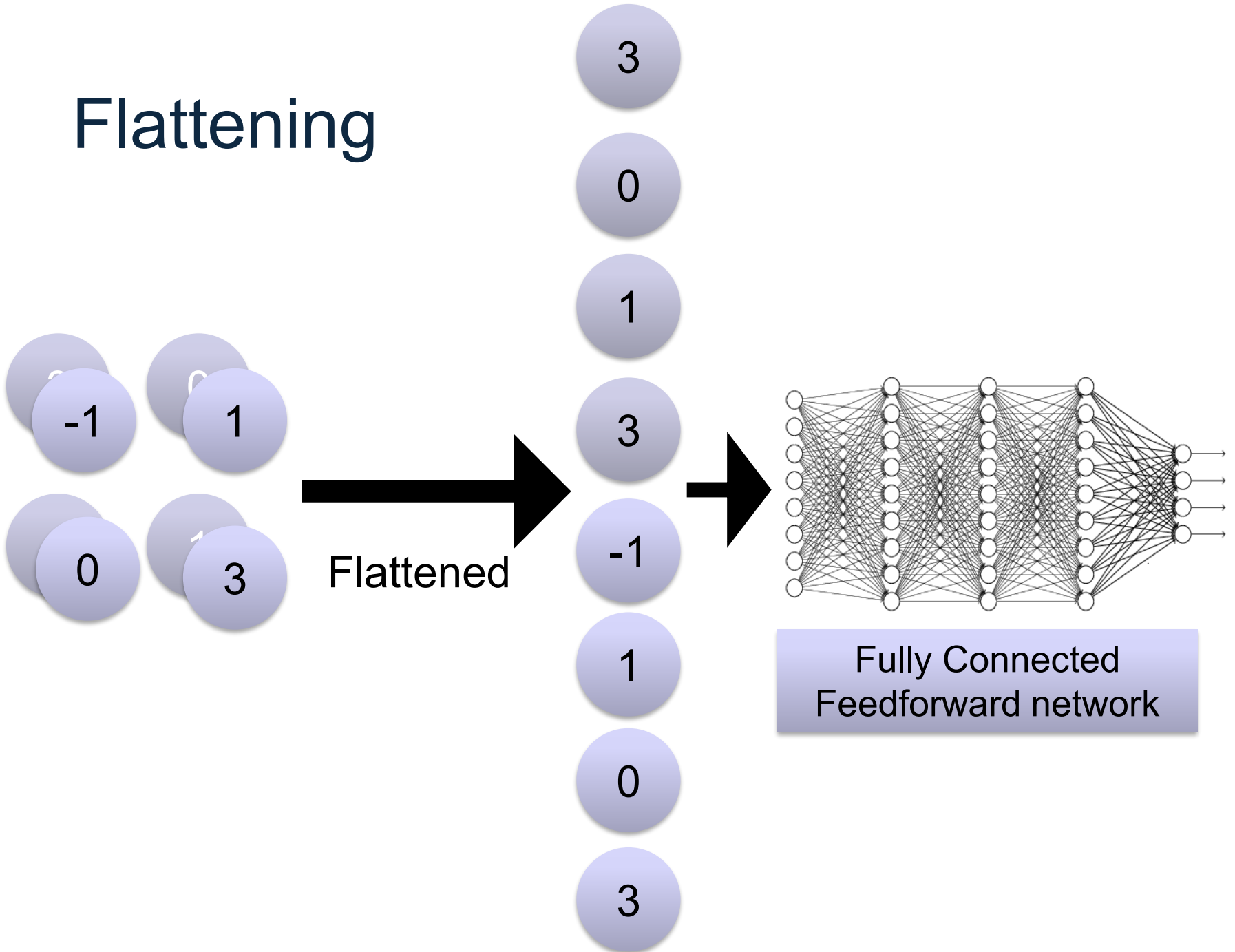
Convolution

Max Pooling

A new image

Flattened

Flattening



CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D tensor)

```
model2.add( Convolution2D( 25, 3, 3,  
                           input_shape=(28, 28, 1)) )
```

1	-1	-1	1	-1
-1	1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1

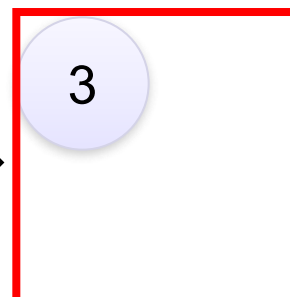
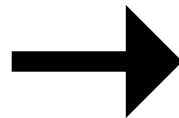
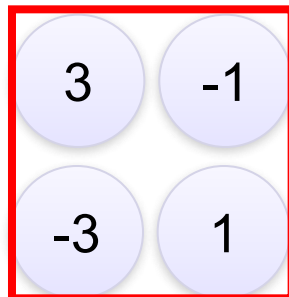
There are
25 3x3
filters.

Input_shape = (28 , 28 , 1)

28 x 28 pixels

1: black/white, 3: RGB

```
model2.add(MaxPooling2D( (2, 2) ))
```



input

Convolution

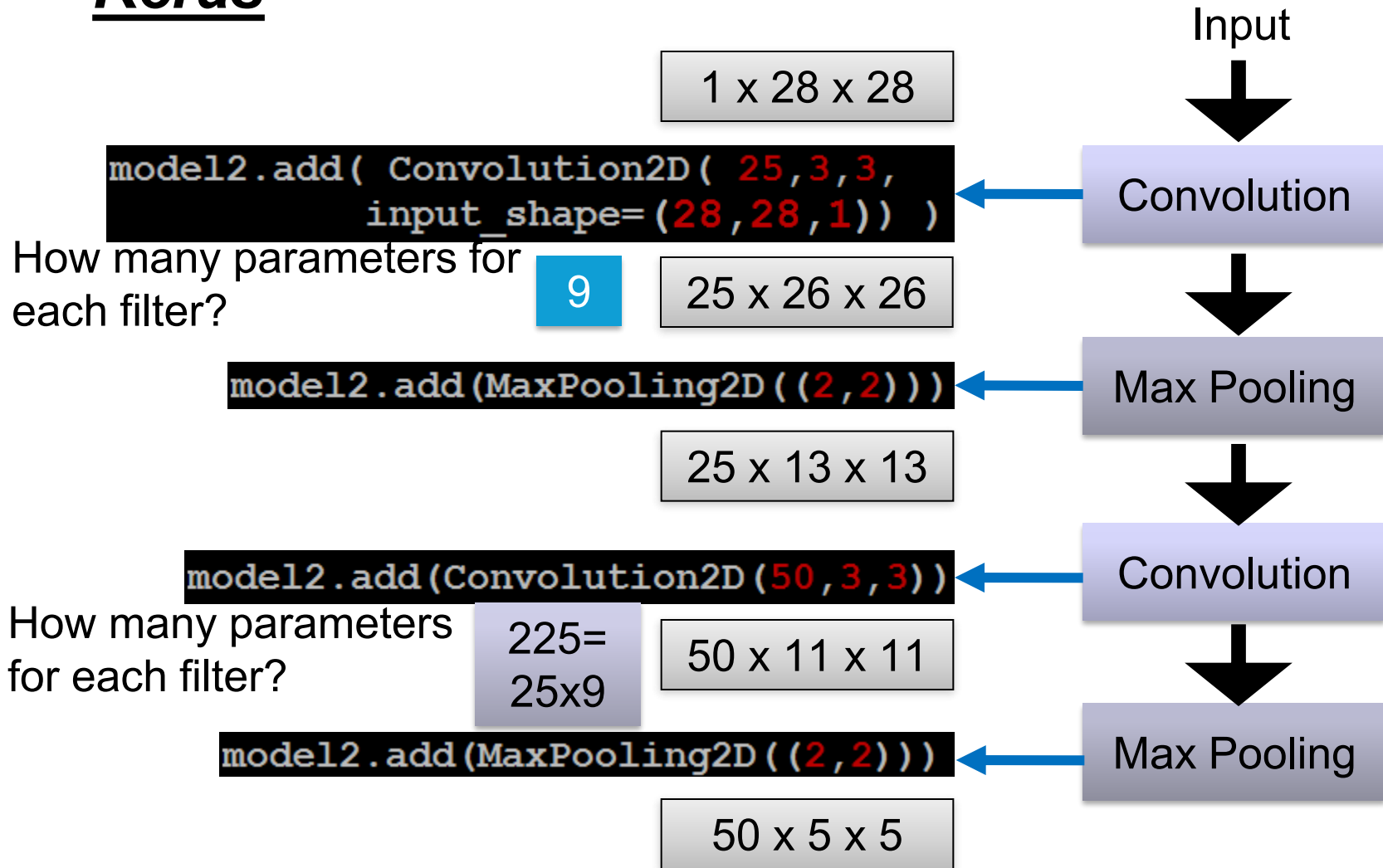
Max Pooling

Convolution

Max Pooling

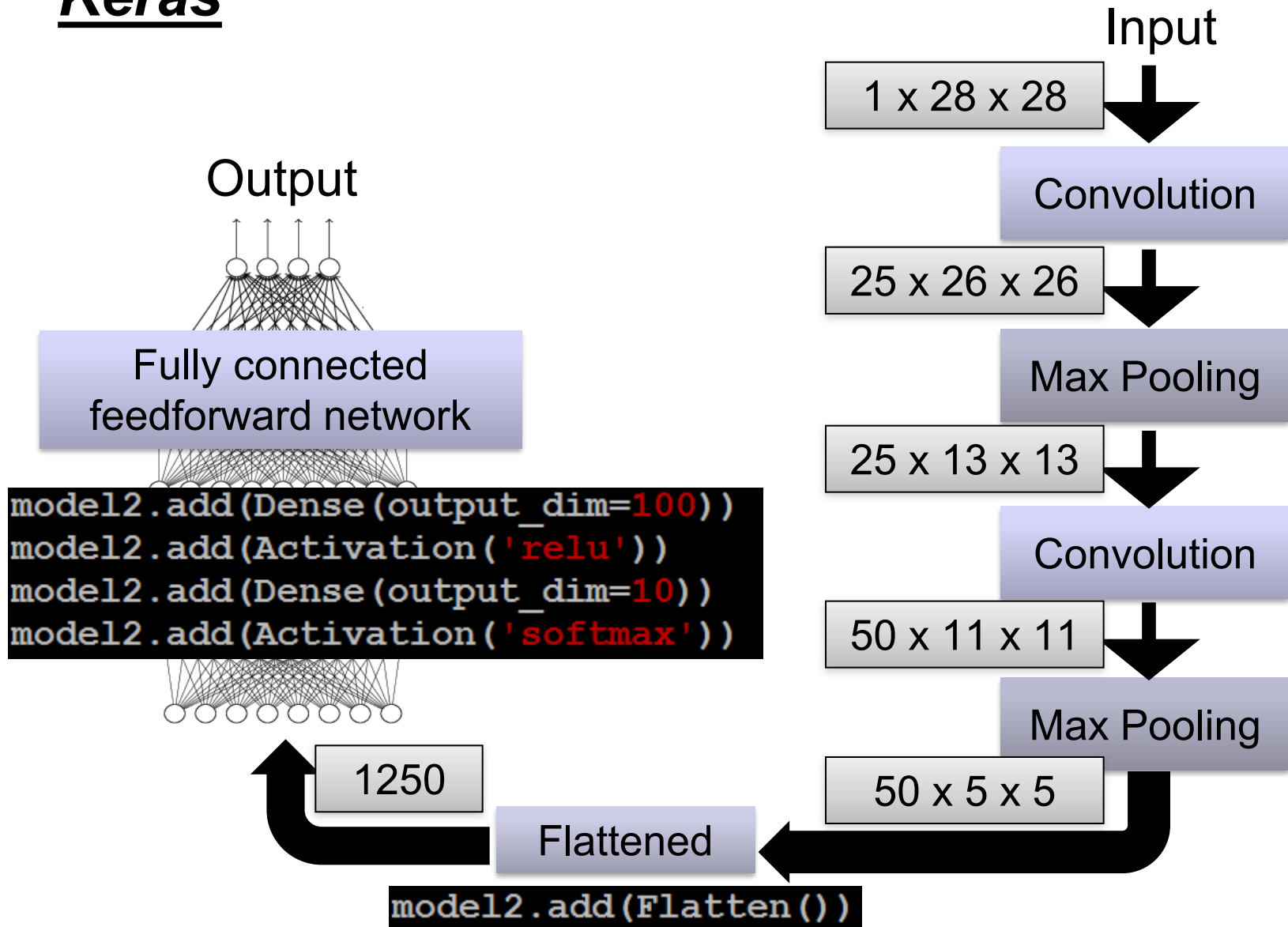
CNN in Keras

Only modified the **network structure** and **input format** (vector -> 3-D array)



CNN in Keras

Only modified the **network structure** and **input format (vector -> 3-D array)**



Parameters/Output Calculation for CNN

Output Size at Convolutional/Pooling Layers

1. Output Size (O):

$$O = \frac{W - K + 2P}{S} + 1$$

- W : Input size (Width/Height)
- K : Filter/Kernel size
- P : Padding
- S : Stride

Number of Parameters (ft. Conv Layers)

2. Number of Parameters (P_{layer}):

$$P_{layer} = [(K \times K \times C_{in}) + 1] \times C_{out}$$

- C_{in} : Input channels (e.g., 3 for RGB)
- C_{out} : Number of filters (Output channels)
- The "+1" represents the **bias** for each filter.

Fully Connected Layer

Step 1: Flatten the output (If connected to conv/pool layer).

Multiply the dimensions: **W**(width) * **H**(height) * **C**(channels)

Step 2:

$$\text{Parameters} = (N_{in} \times N_{out}) + N_{out}$$

Example 1:

Layer #	Layer Type	Specific Configuration
0	Input	Grayscale image
1	Conv2D_1	Filters=6, Kernel=(5,5), Stride=1, Pad=0
2	MaxPool_1	Pool=(2,2), Stride=2
3	Flatten	Reshape to 1D vector
4	Dense_1	Units=120 (Hidden)
5	Dense_2	Units=10 (Output/Digits)

Conv2D_1:

Input: $28 \times 28 \times 1$

Layer 1: Conv2D_1

- **Dimensions:** $O = \frac{28-5+2(0)}{1} + 1 = 24$. Output: $24 \times 24 \times 6$.
- **Parameters:** $[(5 \times 5 \times 1) + 1] \times 6 = 26 \times 6 = \mathbf{156}$.

MaxPool_1:

Layer 2: MaxPool_1

- **Dimensions:** $O = \frac{24-2}{2} + 1 = 12$. Output: $12 \times 12 \times 6$.
- **Parameters:** 0 (Pooling has no weights).

Flatten

Layer 3: Flatten

- **Dimensions:** $12 \times 12 \times 6 = 864$ (1D vector).
- **Parameters:** 0.

Dense_1

Layer 4: Dense_1 (Hidden)

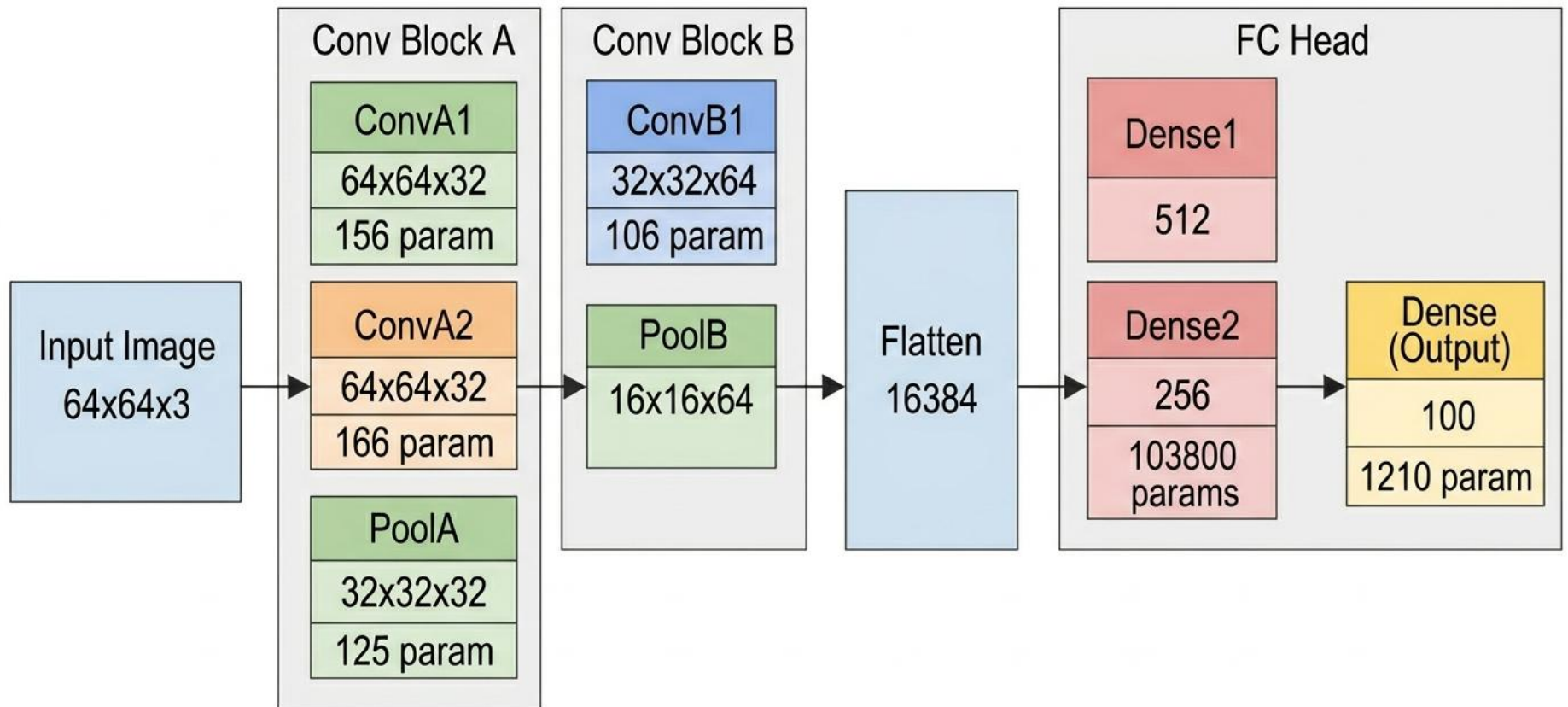
- **Parameters:** $(\text{Input Units} \times \text{Current Units}) + \text{Biases} = (864 \times 120) + 120 = 103,800$.

Output

Layer 5: Dense_2 (Output)

- **Parameters:** $(120 \times 10) + 10 = \mathbf{1,210}$.

Example 02:



Solution

Block A (Initial Processing)

- **ConvA1:** $O = \frac{64-3+2(1)}{1} + 1 = 64$. Output: $64 \times 64 \times 32$.
 - **Params:** $[(3 \times 3 \times 3) + 1] \times 32 = 28 \times 32 = \mathbf{896}$.
- **ConvA2:** $O = \frac{64-3+2(1)}{1} + 1 = 64$. Output: $64 \times 64 \times 32$.
 - **Params:** $[(3 \times 3 \times 32) + 1] \times 32 = 289 \times 32 = \mathbf{9,248}$.
- **MaxPoolA:** $O = \frac{64-2}{2} + 1 = 32$. Output: $32 \times 32 \times 32$.
 - **Params:** **0**.

Solution

Block B (Increased Depth)

- **ConvB1:** $O = \frac{32-3+2(1)}{1} + 1 = 32$. Output: $32 \times 32 \times 64$.
 - **Params:** $[(3 \times 3 \times 32) + 1] \times 64 = 289 \times 64 = \mathbf{18,496}$.
- **MaxPoolB:** $O = \frac{32-2}{2} + 1 = 16$. Output: $16 \times 16 \times 64$.
 - **Params:** 0.

Solution

FC Head (Classification)

- **Flatten:** $16 \times 16 \times 64 = \mathbf{16,384}$.
- **Dense_1:** $(16,384 \times 512) + 512 = \mathbf{8,389,120}$.
- **Dense_2:** $(512 \times 256) + 256 = \mathbf{131,328}$.
- **Dense_Out:** $(256 \times 100) + 100 = \mathbf{25,700}$.